

Transaction Processing, ACID Properties & Concurrency Control

Database Management Systems -- Detailed Notes

1. Introduction to Transaction Processing

A transaction is a single logical unit of work that accesses and possibly modifies the contents of a database. It is a sequence of one or more operations (reads and writes) that must be executed as an atomic unit -- either all operations complete successfully, or none of them take effect.

Example: A bank transfer from Account A to Account B involves two operations: (1) Debit \$1000 from Account A, (2) Credit \$1000 to Account B. Both operations together form a single transaction. If only one succeeds, the database becomes inconsistent.

Transaction States

A transaction moves through the following states during its execution:

- 1 Active - The initial state; the transaction is executing.
- 2 Partially Committed - After the final statement has executed, but before commit.
- 3 Committed - After successful completion; changes are permanently saved.
- 4 Failed - When normal execution can no longer proceed (due to a hardware/logical error).
- 5 Aborted - After the transaction has been rolled back and the database restored to its prior state.
- 6 Terminated - The transaction leaves the system (after commit or abort).

State flow (conceptual):

Active -> Partially Committed -> Committed -> Terminated

Active/Partially Committed -> Failed -> Aborted -> Terminated

Operations on a Transaction

- * Read(X): Reads the value of data item X from the database into a buffer.
- * Write(X): Writes the value from the buffer back into the database.
- * Commit: Signals successful completion; changes become permanent.
- * Rollback/Abort: Undoes all changes made by the transaction.

2. ACID Properties

For a transaction to maintain database consistency and reliability, it must satisfy four key properties: ACID.

A - Atomicity: "All or nothing" principle. A transaction is treated as a single, indivisible unit -- either all its operations are executed, or none are. Managed using transaction logs and rollback mechanisms.

C - Consistency: A transaction must take the database from one valid (consistent) state to another valid state, preserving all integrity constraints (primary keys, foreign keys, triggers).

I - Isolation: Multiple transactions executing concurrently should not interfere with one another. The intermediate state of a transaction should not be visible until commit (achieved via locking and scheduling).

D - Durability: Once committed, changes must persist permanently, even during crashes or power failure (WAL logging).

ACID Summary Table & Concurrent Execution Problems

Property	Ensures	Mechanism Used
Atomicity	All-or-nothing execution	Logging, rollback
Consistency	Valid state transitions	Constraints, triggers
Isolation	No interference between transactions	Locking, scheduling
Durability	Permanence after commit	Write-Ahead Logging (WAL), checkpoints

3. Concurrent Execution & Its Problems

Multiple transactions running simultaneously improve system throughput and resource utilization, but if not controlled properly, they lead to data inconsistency. Common concurrency anomalies include:

Two transactions read the same data item and update it, and one update overwrites the other, causing the first to be lost.

A transaction reads data that has been modified by another transaction that has not yet committed. If T2 aborts, T1 read invalid data.

A transaction reads the same data item twice and gets different values because another transaction modified and committed changes.

A transaction re-executes a query returning a set of rows and finds that the set has changed (rows inserted/deleted by T2).

4. Schedules & Serializability

A schedule is the chronological order in which the operations of multiple transactions are executed.

Types of Schedules:

- 1 Serial Schedule: Transactions execute one after another with no overlapping -- always consistent, but inefficient.
- 2 Concurrent (Non-serial) Schedule: Operations of multiple transactions are interleaved -- improves performance.
- 3 Serializable Schedule: A concurrent schedule that produces the same result as some serial execution.

Serializability & Recoverability:

- * Conflict Serializability: Can be transformed into a serial schedule by swapping non-conflicting operations.
- * View Serializability: Schedule is view-equivalent to some serial schedule (same reads, same final writes).
- * Recoverable Schedule: A transaction commits only after all transactions from which it has read have committed.
- * Cascadeless Schedule: A transaction reads only values written by committed transactions -- avoids cascading aborts.
- * Strict Schedule: A transaction neither reads nor writes data until the last writer commits -- safest form.

5. Concurrency Control Protocols & 6. Deadlocks

Concurrency control ensures concurrent transaction execution leads to a consistent, serializable database state.

5.1 Lock-Based Protocols

- * Shared Lock (S): Allows a transaction to read a data item; multiple transactions can hold S locks simultaneously.
- * Exclusive Lock (X): Allows both read and write; only one transaction can hold X lock at a time.

Two-Phase Locking (2PL) Protocol:

- 1 Growing Phase: The transaction may acquire locks but cannot release any.
- 2 Shrinking Phase: The transaction may release locks but cannot acquire any new ones.

Variants: Strict 2PL (holds exclusive locks until commit/abort); Rigorous 2PL (holds all locks until commit).

5.2 Timestamp-Based Protocols

Each transaction is assigned a unique timestamp $TS(T_i)$ upon arrival. Conflicts are resolved ensuring transactions are ordered as if executed serially by timestamp order. Thomas' Write Rule allows out-of-order obsolete writes to be skipped.

5.3 Validation (Optimistic) Concurrency Control

Assumes conflicts are rare. Executes in three phases: (1) Read Phase (updates local copy), (2) Validation Phase (checks serializability violations), (3) Write Phase (commits changes to database). Best for low-conflict workloads.

5.4 Multiple Granularity Locking

Locks at different hierarchy levels (database → table → page → row) using intention locks (IS, IX) to optimize lock overhead.

6. Deadlocks

A deadlock occurs when two or more transactions are waiting indefinitely for one another to release locks, forming a cycle of dependencies (e.g., T1 holds A, waits for B; T2 holds B, waits for A).

Deadlock Handling Approaches:

- 1 Deadlock Prevention: Wait-Die scheme (older waits, younger dies) or Wound-Wait scheme (older wounds/aborts younger).
- 2 Deadlock Detection: System constructs a Wait-For Graph (WFG); if a cycle is detected, a victim transaction is rolled back.
- 3 Deadlock Avoidance: Uses timestamp ordering or resource allocation graphs to ensure safe states before granting locks.

7. Summary & Conceptual Synthesis

Concept	Key Idea
Transaction	Logical unit of work; sequence of atomic read/write operations
ACID	Atomicity, Consistency, Isolation, Durability -- guarantees reliable transactions
Schedule	Chronological order of execution of concurrent transaction operations
Serializability	Concurrent schedule equivalent to some serial schedule
Concurrency Control	Locking, timestamping, validation to maintain consistency during concurrent execution
Deadlock	Circular wait among transactions; handled via prevention, detection, or avoidance

Why it matters:

Transaction processing and concurrency control together allow modern DBMSs to support multiple users simultaneously without sacrificing data correctness -- this is the backbone of reliable systems like banking applications, airline reservations, and e-commerce platforms worldwide.

AI_Book Digital Campus Notes Repository -- Verified Database Systems Lecture Notes